

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине «Вычислительная математика»
Тема: Решение нелинейных уравнений

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Целью лабораторной работы является изучение методов бисекции, хорд, Ньютона, простых итераций для решения нелинейных уравнений, написание программ, реализующих данные методы с заданной точностью, исследование обусловленности методов (чувствительности к ошибкам в исходных данных).

Задания.

Метод бисекции.

Требуется найти корень уравнения $f(x)=0$ методом бисекции с заданной точностью Eps , исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Выполнение работы осуществляется по индивидуальным вариантам заданий (нелинейных уравнений), приведенным в подразделе 3.6. Номер варианта для каждого студента определяется преподавателем.

Порядок выполнения работы должен быть следующим:

- 1) Графически или аналитически отделить корень уравнения $f(x)=0$ (т.е. найти отрезки $[Left, Right]$, на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).
- 2) Составить подпрограмму вычисления функции $f(x)$.
- 3) Составить головную программу, содержащую обращение к подпрограмме $f(x)$, *BISECT*, *Round* и индикацию результатов.
- 4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps .
- 5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы *Round*, округляющей значения функции с заданной точностью $Delta$.

Метод хорд.

Требуется найти корень уравнения $f(x)=0$ с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленность метода.

Для данной работы задаются индивидуальные варианты нелинейных уравнений (см. подраздел 3.6).

Порядок выполнения работы:

- 1) Графически или аналитически отделить корень уравнения $f(x)=0$ (т.е. найти отрезки $[Left, Right]$, на которых функция $f(x)$ удовлетворяет условиям применимости метода).
- 2) Составить подпрограмму - функцию вычисления функции $f(x)$, предусмотрев округление значений функции с заданной точностью $Delta$ с использованием программы `Round`.
- 3) Составить главную программу, вычисляющую корень уравнения $f(x)=0$ и содержащую обращение к подпрограмме `f(x)`, `HORDA`, `Round` и индикацию результатов.
- 4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.
- 5) В подразделе 3.7 приводится текст программы - функции `HORDA`, предназначенной для решения уравнения $f(x)=0$ методом хорд.

Метод Ньютона.

Используя программы-функции `NEWTON` и `ROUND`, найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции $f(x)$ задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения работы:

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки $[Left, Right]$, на котором функция $f(x)$ удовлетворяет условиям сходимости метода Ньютона).

- 2) Составить подпрограммы - функции вычисления $f(x)$, $f'(x)$, предусмотрев округление их значений с заданной точностью Δ .
- 3) Составить главную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограммам $f(x)$, $f'(x)$, $ROUND$, $NEWTON$ и индикацию результатов.
- 4) Выбрать начальное приближение корня x_0 из $[Left, Right]$ так, чтобы $f(x) * f'(x) > 0$.
- 5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Для приближенного вычисления корней уравнения $f(x) = 0$ методом Ньютона предназначена программа - функция $NEWTON$, текст которой представлен в подразделе 3.7.

Метод простых итераций.

Предлагается, используя программы-функции $ITER$ и $ROUND$, найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом итераций, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции $f(x)$ задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения работы должен быть следующим:

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$.
- 2) Преобразовать уравнение $f(x) = 0$ к виду $x = \varphi(x)$ так, чтобы в некоторой окрестности $[Left, Right]$ корня производная $\varphi'(x)$ удовлетворяла условию $|\varphi'(x)| \leq q < 1$. При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.
- 3) Выбрать начальное приближение, лежащее на $[Left, Right]$.
- 4) Составить подпрограмму для вычисления значений $\varphi(x)$, $\varphi'(x)$, предусмотрев округление вычисленных значений с точностью Δ .

- 5) Составить главную программу, вычисляющую корень уравнения и содержащую обращение к программам $\varphi(x)$, $\varphi'(x)$ и *ITER* и индикацию результатов.
- 6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Текст программы — функции *ITER*, позволяющей вычислять корни уравнения $x = \varphi(x)$ для любой функции, которая удовлетворяет достаточным условиям сходимости метода, представлен в подразделе 3.7.

Вариант 3: $f(x) = \tan x - \frac{1}{x}$

Отделение корней уравнения.

Рассмотрим функцию $f(x) = \tan x - \frac{1}{x}$ и её поведение. Она определена на $\left(x \in \mathbb{R} \mid 0, \frac{\pi}{2} + \pi k, k \in \mathbb{Z}\right)$. Так как функция не является непрерывной на всей области определения, возьмем промежуток $x \in \left(\frac{\pi}{2}; \frac{3\pi}{2}\right)$. Производная равна:

$$f'(x) = \frac{1}{\cos^2 x} + \frac{1}{x^2}$$

Оба слагаемых производной положительны для всех $x \in \left(\frac{\pi}{2}; \frac{3\pi}{2}\right)$, следовательно, $f'(x) > 0$ для всего промежутка. Это значит, что функция монотонно возрастает на данном промежутке.

При $x \rightarrow \frac{\pi}{2}^+ : \tan x \rightarrow -\infty, \frac{1}{x} \rightarrow \frac{2}{\pi}$, поэтому $f(x) \rightarrow -\infty$

При $x \rightarrow \frac{3\pi}{2}^- : \tan x \rightarrow +\infty, \frac{1}{x} \rightarrow \frac{2}{3\pi}$, поэтому $f(x) \rightarrow +\infty$

Функция непрерывна на $x \in \left(\frac{\pi}{2}; \frac{3\pi}{2}\right)$ и меняет знак с $-\infty$ на $+\infty$. По теореме Больцано-Коши корень существует. Так как функция строго возрастает, корень единственный.

Локализуем корень:

$$\text{При } x=3.3: f(3.3)=\operatorname{tg}(3.3)-\frac{1}{3.3} \approx -0,143 < 0$$

$$\text{При } x=3.5: f(3.5)=\operatorname{tg}(3.5)-\frac{1}{3.5} \approx 0,089 > 0$$

Условия теоремы выполняются, следовательно корень локализован на отрезке $[3.3; 3.5]$ (см. рис. 1).

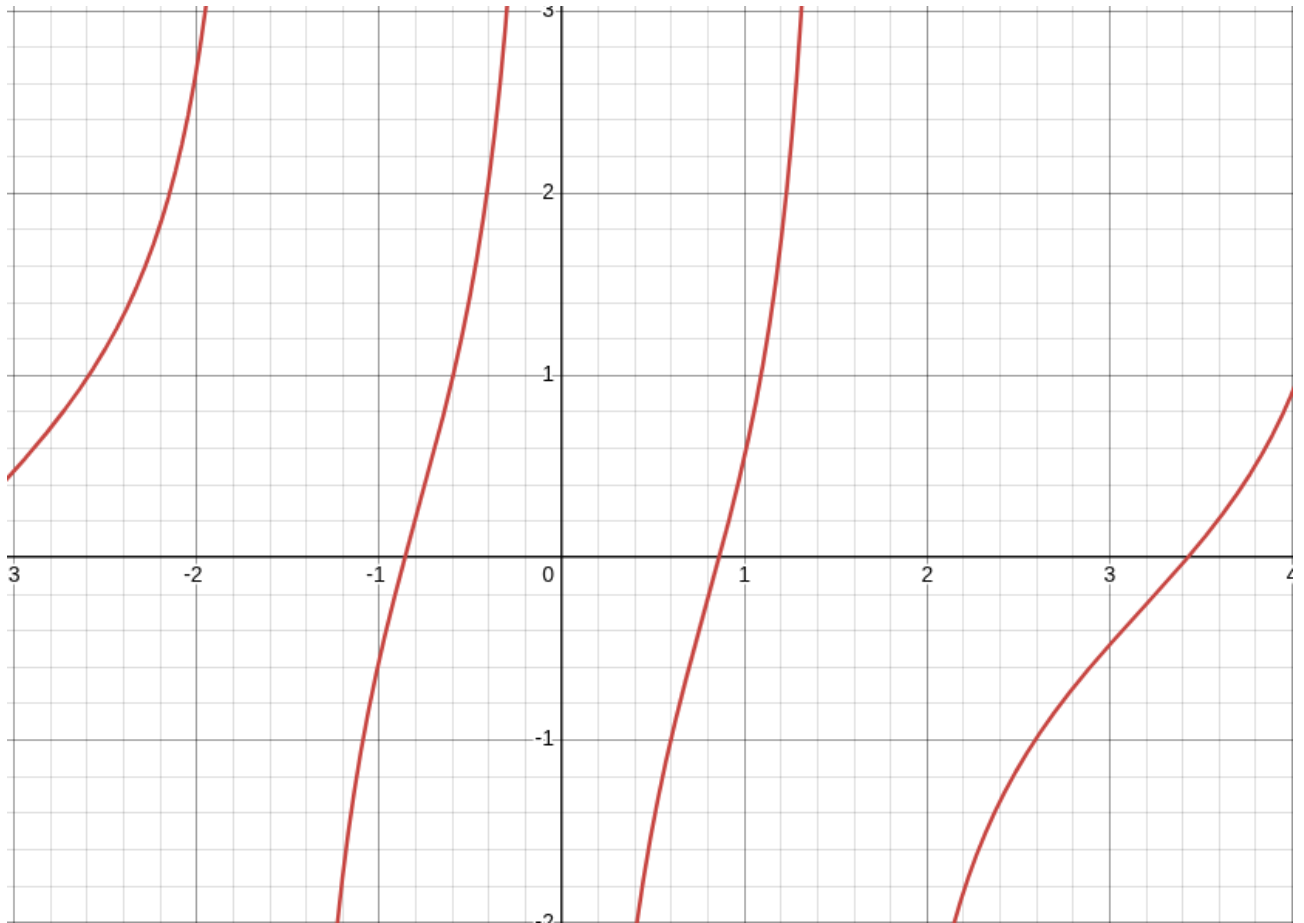


Рисунок 1 — График $f(x)$

Определение начального приближения для метода Ньютона.

1. Найдем вторую и третью производную функции:

$$f''(x) = \frac{2 \sin x}{\cos^3 x} - \frac{2}{x^3}$$

$$f'''(x) = \frac{2 \cos^2 x + 6 \sin^2 x}{\cos^4 x} + \frac{6}{x^4}$$

Оба слагаемых третьей производной $f'''(x)$ положительны для всех x на промежутке $x \in \left(\frac{\pi}{2}; \frac{3\pi}{2}\right)$, значит, вторая производная $f''(x)$ монотонно возрастает на этом промежутке. Так как рассматриваемый отрезок $x \in [3.3; 3.5]$ входит в $x \in \left(\frac{\pi}{2}; \frac{3\pi}{2}\right)$, то поведение $f''(x)$ на нем такое же.

Получили, что $f''(x)$ монотонно возрастает при любом $x \in [3.3; 3.5]$, тогда $f'(x)$ тоже будет монотонно возрастать на этом промежутке.

Построим графики первой и второй производной (см. рис. 2, рис. 3) и убедимся, что на промежутке $x \in [3.3; 3.5]$ функции $f'(x)$ и $f''(x)$ монотонны.

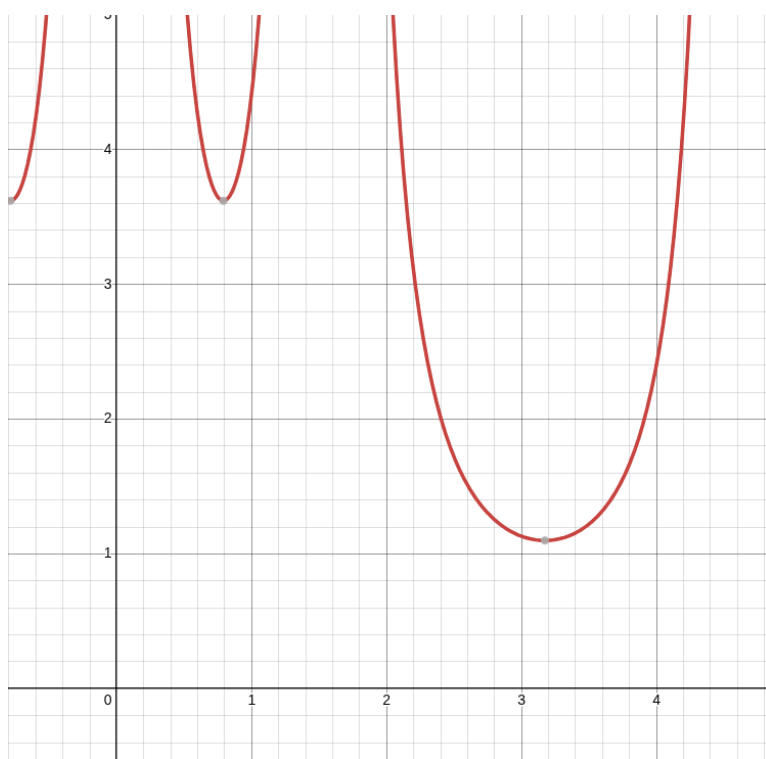


Рисунок 2 — График $f'(x)$

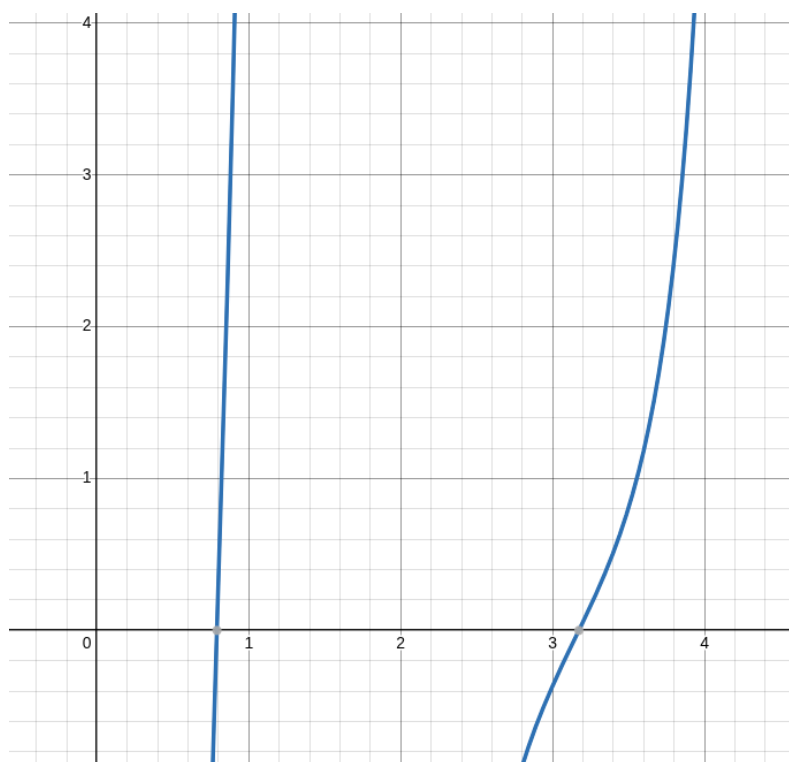


Рисунок 3 — График $f''(x)$

2. По теореме Больцано-Коши корень на отрезке $x \in [3.3; 3.5]$ существует (доказано выше при выборе начального отрезка).
3. Производная $f'(x)$ не обращается в 0 на отрезке $x \in [3.3; 3.5]$, так как она монотонно возрастает на нем.
4. Рассмотрим $x_0 = 3.5$:

$$f'(x_0) = \operatorname{tg}(3.5) - \frac{1}{3.5} = 0,089 > 0$$

$$f''(x_0) = \frac{2 \sin(3.5)}{\cos^3(3.5)} - \frac{2}{(3.5)^3} = 0,801 > 0$$

Условия выполняются, поэтому начальное приближение $x_0 = 3.5$.

Преобразование уравнения $f(x)=0$ к виду $x=\varphi(x)$ для метода простых итераций.

Пусть $\varphi(x) = x - \alpha f(x)$. Тогда $x = x - \alpha \cdot f(x)$, следовательно $f(x)=0$. Значит уравнение $x=\varphi(x)$ эквивалентно $f(x)=0$ при $\alpha \neq 0$.

Подберем коэффициент α . Условие сходимости метода простых итераций $|\phi'(x)| \leq q < 1$, где q — максимальное значение производной на отрезке [Left, Right]. Для наиболее быстрой сходимости значение q должно быть минимальным. Найдем производную: $\phi'(x) = 1 - \alpha f'(x)$.

Производная $f'(x) > 0$, значит существуют $m, M > 0$, такие что $0 < m \leq |f'(x)| \leq M$.

Тогда $1 - \alpha M \leq |\phi'(x)| \leq 1 - \alpha m$. Для минимизации максимального модуля производной необходимо расположить крайние значения симметрично относительно нуля: $1 - \alpha M = -(1 - \alpha m)$. Получаем $\alpha = \frac{2}{m+M}$.

Найдем максимальное и минимальное значение производной:

$$M = f'(3.5) = \frac{1}{\cos^2(3.5)} + \frac{1}{(3.5)^2} \approx 1,222$$

$$m = f'(3.3) = \frac{1}{\cos^2(3.3)} + \frac{1}{(3.3)^2} \approx 1,117$$

По полученным значениям m и M найдем коэффициент α :

$$\alpha = \frac{2}{m+M} = \frac{2}{1.117+1.222} = \frac{2}{2.339} = 0.855$$

Итоговая формула перехода имеет вид:

$$\phi'(x) = x - 0.855 * f(x)$$

Начальное приближение $x_0 = 3.5$. Это значение принадлежит рассматриваемому промежутку и находится близко к корню, что позволяет уменьшить количество итераций.

Разработка программы.

Программа, реализующая каждый из методов, написана на языке программирования C++.

Функция `double f(double x)` вычисляет значение функции $f(x)$:

```
double f(double x){
    return tan(x) - 1/x;
}
```

Функция *double f1(double x)* вычисляет значение производной функции $f(x)$:

```
double f1(double x){  
    return 1/(cos(x)*cos(x)) + 1/(x*x);  
}
```

Функция *double F(double x)* вычисляет значение функции:

```
double F(double x){  
    return x - (f(x) / 2);  
}
```

Функция *double bisect(double Left, double Right, double Eps, int &N)* реализует метод бисекции, итеративно сокращая длину отрезка в 2 раза, пока она не станет меньше , а также подсчитывает количество итераций для вычисления корня при заданной точности.

Принимаемые аргументы:

- *double Left* — левая граница отрезка;
- *double Right* — правая граница отрезка;
- *double Eps* — точность вычисления корня;
- *int &N* — количество итераций, требуемое для вычисления корня.

Возвращаемое значение:

- *double root* — искомый корень.

Функция *double horda(double Left, double Right, double Eps, int &N)* реализует метод хорд, итеративно приближает корень, пока значение функции в текущем приближении не станет меньше заданной точности эпсилон.

Принимаемые аргументы:

- *double Left* — левая граница отрезка;
- *double Right* — правая граница отрезка;
- *double Eps* — точность вычисления корня;

- *int &N* — количество итераций, требуемое для вычисления корня.

Возвращаемое значение:

- *double root* — искомый корень.

Функция *double newton(double x, double eps, int &cntIter)* реализует метод Ньютона и подсчитывает число итераций, необходимых для достижения заданной точности.

Принимаемые аргументы:

- *double x* — начальное приближение корня;
- *double eps* — точность вычисления;
- *int &cntIter* — ссылка на переменную-счетчик итераций.

Возвращаемое значение:

- *double res* — корень уравнения.

Функция *double iter(double x0, double eps, int &n)* реализует метод простых итераций и подсчитывает число итераций, необходимых для достижения заданной точности.

Принимаемые аргументы:

5. *double x0* — начальное приближение корня;
6. *double eps* — точность вычисления;
7. *int &n* — ссылка на переменную-счетчик итераций.

Возвращаемое значение:

8. *double res* — корень уравнения.

Функция *double Round(double X, double Delta)* округляет значение *X* с точностью *Delta*.

Принимаемые аргументы:

- *double X* — значение для округления;
- *double Delta* — точность округления.

Возвращаемое значение:

- *double val* — округленное значение.

Исходный код находится в **приложении А**.

Исследование зависимости числа итераций от точности вычисления эпсилон для метода бисекции.

В функции *BISECT* производился подсчет числа итераций для каждой точности вычисления.

Опираясь на график, приведенный ниже (см. рис. 2), можно сделать вывод, что при увеличении числа знаков в дробной части эпсилон возрастает количество итераций. Это объясняется тем, что на каждой итерации рассматриваемый отрезок делится пополам (или умножается на 0.5), что добавляет ему только один знак в дробную часть. Чем больше знаков в дробной части эпсилон, тем больше итераций нужно сделать, чтобы достичь желаемой точности вычисления.

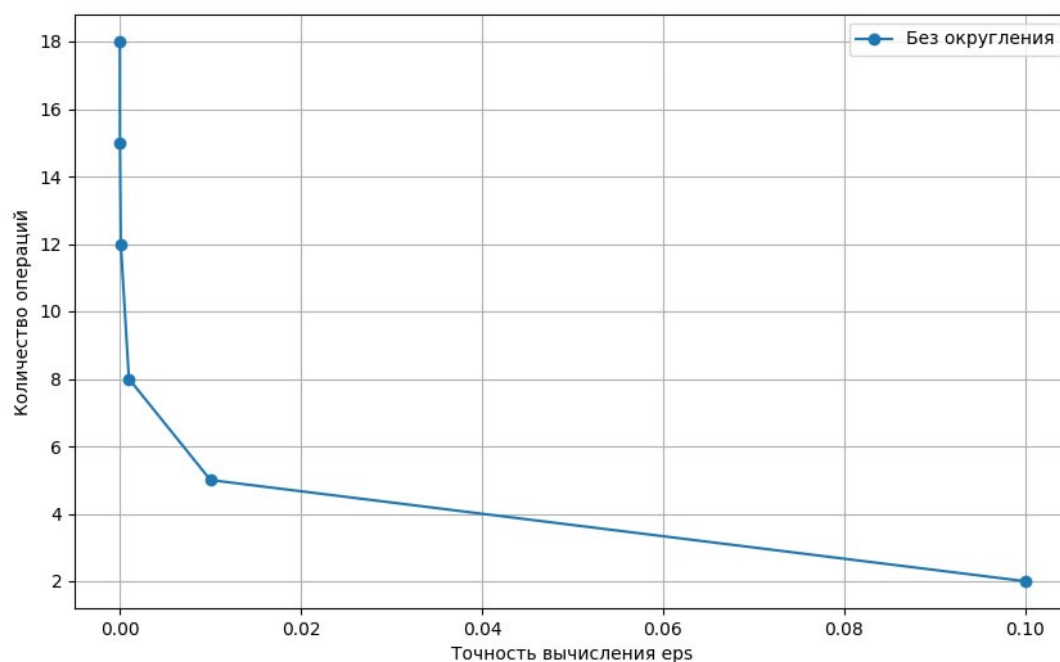


Рисунок 2 — График зависимости числа операций от ϵ_{ps}

Исследование скорости сходимости.

Метод бисекции.

Для метода бисекции скорость сходимости является линейной. Длина отрезка, содержащего корень, на каждой итерации уменьшается вдвое. Если начальный отрезок $[a_0, b_0]$, то после n итераций:

$$|x_n - x^*| \leq \frac{b_0 - a_0}{2^{(n+1)}}$$

Таким образом, для достижения точности ε требуется:

$$n \leq \log_2 \left(\frac{b_0 - a_0}{\varepsilon} \right) - 1$$

Метод гарантирует сходимость, но медленнее квадратично сходящихся методов.

Метод хорд.

Скорость сходимости для метода хорд вычисляется по формуле:

$$C \approx \frac{|x_{n+1} - x_0|}{|x_n - x_0|},$$

где x_{n+1} - значение корня на текущей итерации, x_n - значение корня на предыдущей итерации, x_0 - значение корня, наиболее близкое к действительному. Возьмем $x_0 = 0.860334$ при точности вычисления $eps = 1e-6$ как эталонное.

Рассмотрим последовательно каждую итерацию для $eps = 1e-5$ и ее скорость сходимости (см. рис. 4). По графику видно, что скорость сходимости уменьшается на каждом шаге, следовательно, метод сходится.

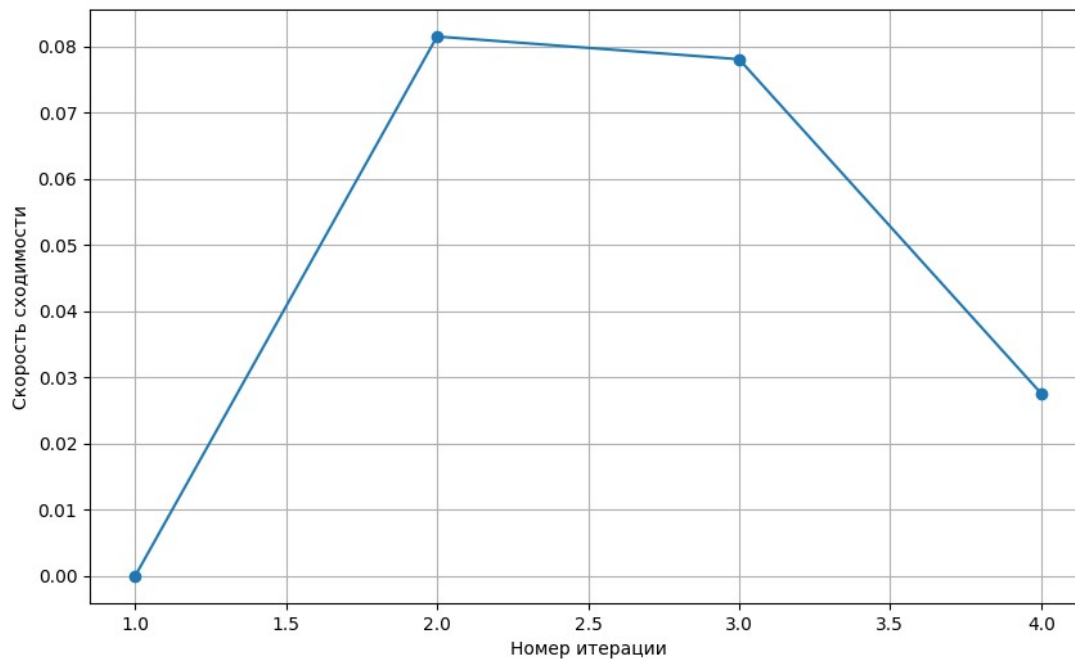


Рисунок 4 — График сходимости метода хорд

Метод Ньютона.

Для оценки скорости сходимости проанализируем зависимость числа итераций, требуемую для вычисления корня, от заданной вычислительной точности.

График (см. рис. 5) показывает незначительный рост итераций для $1e-1 \leq \epsilon \leq 1e-6$.

Согласно методу Ньютона, число верных знаков на каждой итерации увеличивается вдвое (квадратичная сходимость). Это значит, что при повышении требований к точности количество итераций вырастет незначительно, что подтверждает эффективность метода.

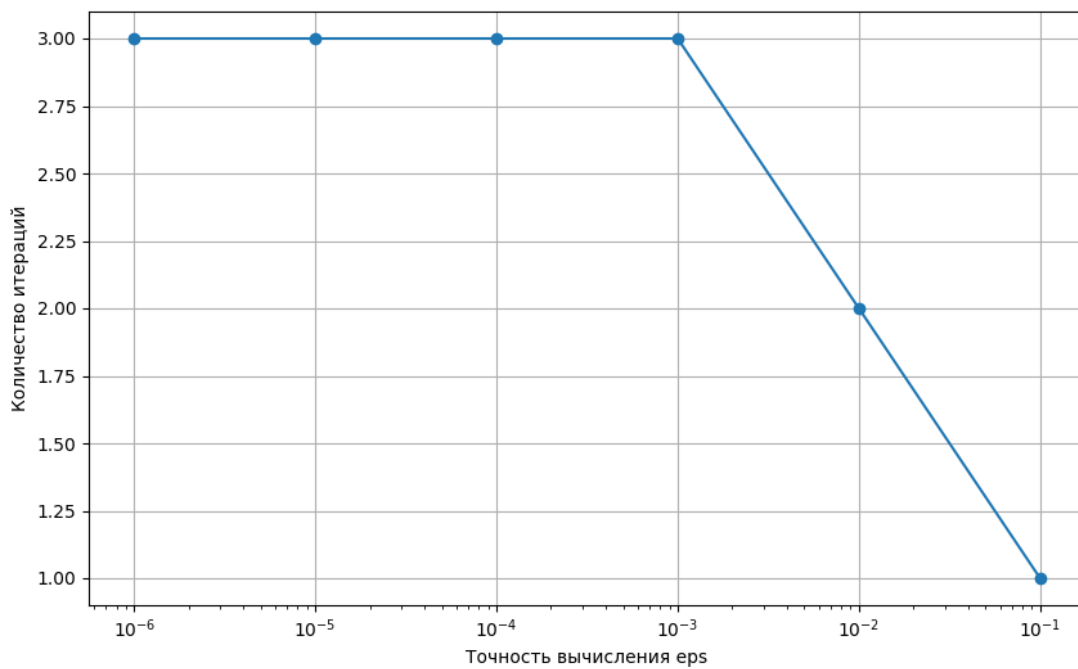


Рисунок 5 — График зависимости количества итераций от точности вычисления

Метод простых итераций.

В теории метод простых итераций имеет линейную скорость сходимости, что подтверждает график (см. рис. 2), который показывает примерно линейный рост итераций для $1e-1 \leq \epsilon \leq 1e-6$.

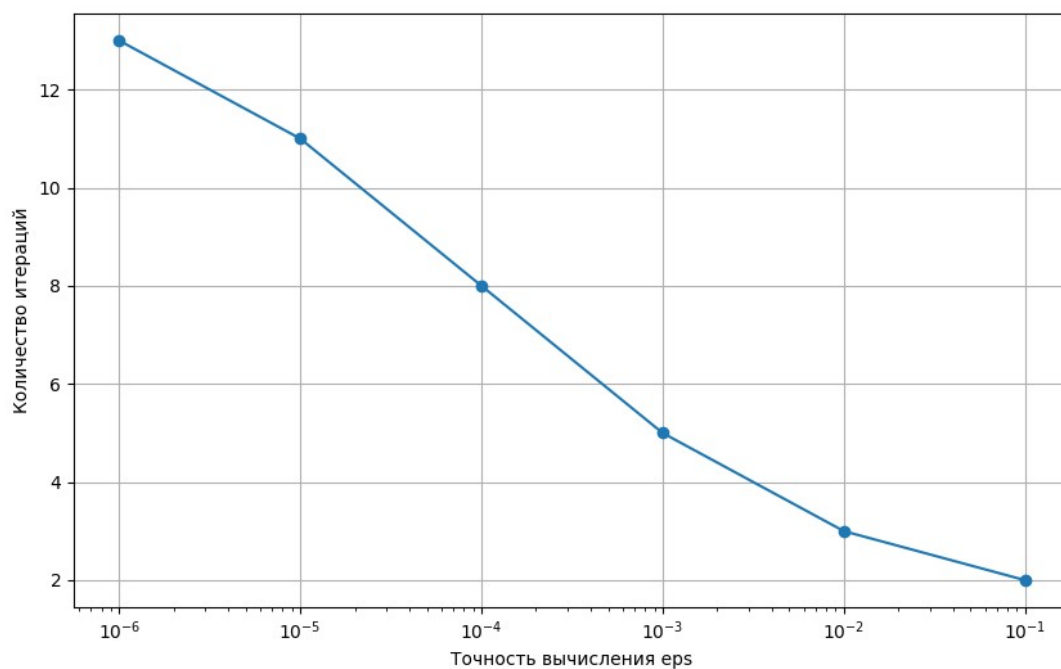


Рисунок 2 — График зависимости количества итераций от точности вычисления

Тестирование.

В таблице 1 приведены результаты работы программы для различных значений эпсилон.

Таблица 1 - Результаты тестирования

Метод	Входные данные	Выходные данные
Метод бисекции	0.1	3.4
	0.01	3.44
	0.001	3.427
	0.0001	3.4256
	0.00001	3.42562
	0.000001	3.425618
Метод хорд	0.1	3.4
	0.01	3.42
	0.001	3.426
	0.0001	3.4256
	0.00001	3.42562
	0.000001	3.425618

Метод Ньютона	0.1	3.4
	0.01	3.43
	0.001	3.426
	0.0001	3.4256
	0.00001	3.42562
	0.000001	3.425618
Метод простых итераций	0.1	3.4
	0.01	3.43
	0.001	3.426
	0.0001	3.4257
	0.00001	3.42562
	0.000001	3.425619

Чувствительность метода к ошибкам в исходных данных.

Для исследования обусловленности методов использовалась функция *Round*, которая производила округление полученного значения функции с точностью delta для симуляции ошибок в исходных данных. Результаты показаны в таблицах 2-5.

Таблица 2 - Исследование обусловленности метода бисекции

Eps	Delta	Root
0.1	0.1	3.40
	0.01	3.400
	0.001	3.4000
	0.0001	3.40000
	0.00001	3.400000
	0.000001	3.4000000
	0.0000001	3.40000000
	0.00000001	3.400000000
0.01	0.1	3.400
	0.01	3.4400
	0.001	3.43700
	0.0001	3.437500
	0.00001	3.4375000
	0.000001	3.43750000
	0.0000001	3.437500000
	0.00000001	3.4375000000

	0.00000001	3.4375000000
0.001	0.1	3.4000
	0.01	3.43000
	0.001	3.427000
	0.0001	3.4266000
	0.00001	3.42656000
	0.000001	3.426562000
	0.0000001	3.4265625000
	0.00000001	3.42656250000
0.0001	0.1	3.40000
	0.01	3.430000
	0.001	3.4260000
	0.0001	3.42560000
	0.00001	3.425590000
	0.000001	3.4255860000
	0.0000001	3.42558590000
	0.00000001	3.425585940000
0.00001	0.1	3.400000
	0.01	3.4300000
	0.001	3.42600000
	0.0001	3.425600000
	0.00001	3.4256200000
	0.000001	3.42562300000
	0.0000001	3.425622600000
	0.00000001	3.4256225600000
0.000001	0.1	3.4000000
	0.01	3.43000000
	0.001	3.426000000
	0.0001	3.4256000000
	0.00001	3.42562000000
	0.000001	3.425618000000
	0.0000001	3.4256179800000
	0.00000001	3.42561798000002

Таблица 3 - Исследование обусловленности метода хорд

Eps	Delta	Root
0.1	0.1	3.40
	0.01	3.420
	0.001	3.4230
	0.0001	3.42340
	0.00001	3.423440
	0.000001	3.4234380
	0.0000001	3.42343820
	0.00000001	3.423438210
0.01	0.1	3.400
	0.01	3.4200
	0.001	3.42300
	0.0001	3.423400
	0.00001	3.4234400
	0.000001	3.42343800
	0.0000001	3.423438200
	0.00000001	3.4234382100
0.001	0.1	3.4000
	0.01	3.43000
	0.001	3.426000
	0.0001	3.4256000
	0.00001	3.42557000
	0.000001	3.425574000
	0.0000001	3.4255741000
	0.00000001	3.42557410000
0.0001	0.1	3.40000
	0.01	3.430000
	0.001	3.4260000
	0.0001	3.42560000
	0.00001	3.425570000
	0.000001	3.4255740000
	0.0000001	3.42557410000
	0.00000001	3.425574100000
0.00001	0.1	3.400000

	0.01	3.4300000
	0.001	3.42600000
	0.0001	3.425600000
	0.00001	3.4256200000
	0.000001	3.42561800000
	0.0000001	3.425617600000
	0.00000001	3.4256175500000
0.000001	0.1	3.4000000
	0.01	3.43000000
	0.001	3.426000000
	0.0001	3.4256000000
	0.00001	3.42562000000
	0.000001	3.425618000000
	0.0000001	3.4256184000000
	0.00000001	3.42561844000000

Таблица 4 - Исследование обусловленности метода Ньютона

Eps	Delta	Root
0.1	0.1	3.40
	0.01	3.430
	0.001	3.4270
	0.0001	3.42730
	0.00001	3.427270
	0.000001	3.4272710
	0.0000001	3.42727070
	0.00000001	3.427270700
0.01	0.1	3.400
	0.01	3.4300
	0.001	3.42600
	0.0001	3.425600
	0.00001	3.4256200
	0.000001	3.42561900
	0.0000001	3.425619100
	0.00000001	3.4256191400

0.001	0.1	3.4000
	0.01	3.43000
	0.001	3.426000
	0.0001	3.4256000
	0.00001	3.42562000
	0.000001	3.425618000
	0.0000001	3.4256185000
	0.00000001	3.42561846000
0.0001	0.1	3.40000
	0.01	3.430000
	0.001	3.4260000
	0.0001	3.42560000
	0.00001	3.425620000
	0.000001	3.4256180000
	0.0000001	3.42561850000
	0.00000001	3.425618460000
0.00001	0.1	3.400000
	0.01	3.4300000
	0.001	3.42600000
	0.0001	3.425600000
	0.00001	3.4256200000
	0.000001	3.42561800000
	0.0000001	3.425618500000
	0.00000001	3.4256184600000
0.000001	0.1	3.4000000
	0.01	3.43000000
	0.001	3.426000000
	0.0001	3.4256000000
	0.00001	3.42562000000
	0.000001	3.425618000000
	0.0000001	3.4256185000000
	0.00000001	3.42561846000000

Таблица 5 - Исследование обусловленности метода простых итераций

Eps	Delta	Root
0.1	0.1	3.40
	0.01	3.440
	0.001	3.4380
	0.0001	3.43790
	0.00001	3.437900
	0.000001	3.4379020
	0.0000001	3.43790230
	0.00000001	3.437902340
0.01	0.1	3.400
	0.01	3.4300
	0.001	3.43100
	0.0001	3.430700
	0.00001	3.4306900
	0.000001	3.43069100
	0.0000001	3.430691200
	0.00000001	3.4306911600
0.001	0.1	3.4000
	0.01	3.43000
	0.001	3.426000
	0.0001	3.4265000
	0.00001	3.42649000
	0.000001	3.426489000
	0.0000001	3.4264890000
	0.00000001	3.42648898000
0.0001	0.1	3.40000
	0.01	3.430000
	0.001	3.4260000
	0.0001	3.42570000
	0.00001	3.425680000
	0.000001	3.4256810000
	0.0000001	3.42568060000
	0.00000001	3.425680550000
0.00001	0.1	3.400000

	0.01	3.4300000
	0.001	3.42600000
	0.0001	3.425600000
	0.00001	3.4256200000
	0.000001	3.42562300000
	0.0000001	3.425622900000
	0.00000001	3.4256228900000
0.000001	0.1	3.4000000
	0.01	3.43000000
	0.001	3.426000000
	0.0001	3.4256000000
	0.00001	3.42562000000
	0.000001	3.425619000000
	0.0000001	3.4256192000000
	0.00000001	3.42561922000000

Для каждого метода выберем значения, которые наиболее близки к реальным. Возьмем значения без симуляции ошибок в исходных данных при $\epsilon = 1e-6$:

- Метод бисекции — 3.425618;
- Метод хорд — 3.425618;
- Метод Ньютона — 3.425618;
- Метод простых итераций — 3.425619.

Проанализировав таблицы 2-5, можно заметить, что для каждого метода при увеличении точности округления δ уменьшается ошибка в результате.

Также из таблиц видно, что при небольших ошибках в исходных данных меняются только 2-3 последние цифры результата. Это значит, что метод устойчив к ошибкам в исходных данных и дает относительно точные результаты.

Вывод

В результате выполнения лабораторной работы были изучены методы бисекции, хорд, Ньютона, простых итераций для решения нелинейных уравнений; написаны программы на языке C++, реализующие алгоритм для различных значений точности вычислений. Были исследованы скорости сходимости и обусловленность методов. Установлено, что все методы устойчивы к ошибкам входных данных и выдают относительно точный результат.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

```
#include <iostream>
#include <cmath>
#include <fstream>
#include <iomanip>
#include <vector>
#include <string>

double f(double x){
    return tan(x) - 1/x;
}

double f1(double x){    //Производная
    return 1/(cos(x)*cos(x)) + 1/(x*x);
}

double F(double x){    //Фи для простых итераций
    return x - 0.855*f(x);
}

double bisect(double Left, double Right, double Eps, int &N){
    double E = std::abs(Eps)*2.0;
    double FLeft = f(Left);
    double FRight = f(Right);
    double X = (Left + Right) / 2.0;
    double Y;

    if (FLeft * FRight > 0.0) {
        std::cout << "Неверное задание интервала\n";
        exit(1);
    }

    if (Eps <= 0.0) {
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

    N = 0;

    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    while ((Right - Left) >= E){
        X = 0.5*(Right + Left);
        Y = f(X);

        if (Y == 0.0) return (X);

        if (Y*FLeft < 0.0)
            Right = X;
        else {
            Left = X;
            FLeft = Y;
        }
    }
}
```

```

        }
        N++;
    }
    return(X);
}

double horda(double Left, double Right, double Eps, int &N){
    double FLeft = f(Left);
    double FRight = f(Right);
    double X,Y;

    if (FLeft * FRight > 0.0) {
        std::cout << "Неверное задание интервала\n";
        exit(1);
    }

    if (Eps <= 0.0) {
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

    N = 0;

    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    do{
        X = Left - (Right - Left)*FLeft / (FRight - FLeft);
        Y = f(X);

        if (Y == 0.0) return (X);

        if (Y*FLeft < 0.0){
            Right = X;
            FRight = Y;
        } else {
            Left = X;
            FLeft = Y;
        }
        N++;
    } while ( fabs(Y) >= Eps );

    return(X);
}

double newton(double x, double eps, int &cntIter){
    double y, y1, dx;
    cntIter = 0;

    do {
        y = f(x);
        if (y == 0.0)
            return (x);
    }

```

```

        y1 = f1(x);

        if (y1 == 0.0){
            std::cout << "Производная обратилась в ноль\n";
            exit(1);
        }

        dx = y / y1;
        x = x - dx;
        cntIter++;

    } while (fabs(dx) > eps);

    return x;
}

double iter(double x0, double eps, int& n){
    if (eps <= 0.0){
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

    double x1 = F(x0);
    double x2 = F(x1);

    n = 2;

    while((x1 - x2)*(x1 - x2) > fabs((2*x1 - x0 - x2) * eps)){
        x0 = x1;
        x1 = x2;
        x2 = F(x1);
        n++;
    }

    return(x2);
}

double Round(double x, double delta){
    if (delta <= 1E-9) {
        std::cout << "Неверное задание точности округления\n";
        exit(1);
    }

    if (x > 0.0)
        return (delta * (long((x/delta) + 0.5)));
    else
        return (delta * (long((x/delta) - 0.5)));
}

int main(){
    int N;
    double left = 3.3; //Для бисекции и хорд
    double right = 3.5;
    double x0 = 3.5; //Для Ньютона и простых итераций

```

```

std::vector<std::string> methods = {
    "===== МЕТОД БИСЕКЦИИ =====\n\n",
    "===== МЕТОД ХОРД =====\n\n",
    "===== МЕТОД НЬЮТОНА =====\n\n",
    "===== МЕТОД ПРОСТЫХ ИТЕРАЦИЙ =====\n\n"
};

int switcher;
std::cout << "Введите номер метода (0-3)\n";
std::cin >> switcher;

std::ofstream file("results.txt");
file << methods[switcher];
file << "Количество итераций\tТочность вычисления\tТочность
округления\tКорень\n";

for (double e = 0.1; e >= 1E-9; e *= 0.1){
    int cntSignesE = static_cast<int>(-log10(e));
    double root;

    file << std::fixed << std::setprecision(cntSignesE);

    if (switcher == 0){
        root = bisect(left, right, e, N);
        file << N << '\t' << e << '\t' << 0 << '\t' << root <<
'\n';

    } else if (switcher == 1){
        root = horda(left, right, e, N);
        file << N << '\t' << e << '\t' << 0 << '\t' << root <<
'\n';

    } else if (switcher == 2){
        root = newton(x0, e, N);
        file << N << '\t' << e << '\t' << 0 << '\t' << root <<
'\n';

    } else if (switcher == 3){
        root = iter(x0, e, N);
        file << N << '\t' << e << '\t' << 0 << '\t' << root <<
'\n';

    }

    for (double delta = 0.1; delta > 1E-9; delta *= 0.1){
        double rootWithRound = Round(root, delta);

        int cntSignesDelta = static_cast<int>(-log10(delta));
        file << std::fixed << std::setprecision(cntSignesE);
        file << N << '\t' << e << '\t';
        file << std::fixed <<
std::setprecision(cntSignesDelta);
        file << delta << '\t';
        file << std::fixed <<
std::setprecision(cntSignesE+cntSignesDelta);
        file << rootWithRound << '\n';

    }
}

```

```
    }  
    file.close();  
    return 0;  
}
```